1. Banco de Dados Não-Convencional

11.1. Visão Geral: Banco de dados não convencional

Em continuidade ao capítulo anterior, este capítulo detalha o projeto físico da porção do RecarregaVest armazenada em um banco de dados não-convencional (NoSQL). Adotou-se a persistência poliglota: o MySQL (seção 10) mantém o núcleo transacional do sistema (cadastro de usuários e perfis, validação de CRP, sessões, vagas e demais dados relacionais), e o MongoDB armazena apenas duas entidades do modelo conceitual: Postagem (feed das comunidades) e Checklist (checklist diário de autocuidado do vestibulando). Essas entidades foram escolhidas por serem dados de alto volume de escrita, acessados sempre por uma mesma chave de consulta e sem necessidade de junções ou transações com as demais tabelas, o que é o perfil em que um banco orientado a documentos traz vantagem. Este capítulo define o SGBD adotado, apresenta o mapeamento das duas entidades para coleções de documentos, registra as decisões de projeto físico, disponibiliza o script de criação das coleções com chaves, índices e restrições, as operações que substituem as chaves estrangeiras entre os dois bancos, a carga inicial e os diagramas da estrutura física dos dados. As regras de negócio e os casos de uso que dependem dessas entidades (UC05 e UC09) permanecem os mesmos.

11.2. SGBD Utilizado

O projeto utiliza o **MongoDB**, na edição Community Server, como Sistema de Gerenciamento de Banco de Dados não relacional. O MongoDB é orientado a documentos: os dados são armazenados como documentos BSON (representação binária de JSON) agrupados em coleções, sem a necessidade de um esquema fixo, mas com a possibilidade de impor regras de validação por coleção. As especificações necessárias para o correto funcionamento do banco são:

Item	Especificação
Produto	MongoDB Community Server
Versão	8.0 (versão mínima recomendada: 7.0)
Modelo de Dados	Orientado a documentos (BSON)
Engine de armazenamento	WiredTiger (padrão), com compressão snappy nas coleções
Banco de Dados	recarregavest nosql
Linguagem de definição e manipulação	MongoDB Query Language executada no mongosh (JavaScript)
Topologia	Replica set rs0 (necessário para transações multi-documento e alta disponibilidade)

Porta e autenticação	27017; autenticação SCRAM com controle de acesso habilitado (authorization: enabled)
Conexão com a aplicação	Driver oficial mongodb para Node.js (API REST da seção 8)

A escolha do MongoDB se justifica pelos seguintes motivos:

- **Modelo de documentos:** os itens do checklist pertencem a um único vestibulando em um único dia, têm cardinalidade pequena e são lidos e gravados em conjunto; embuti-los em um documento substitui várias linhas e dispensa junção.
- **Escalabilidade e aderência à arquitetura:** particionamento horizontal (sharding) por comunidade atende ao crescimento do número de usuários (RNF07 e RNF10), o driver oficial integra-se à API REST em Node.js e a edição Community é gratuita, compatível com o caráter voluntário do projeto.
- **Alto volume de escrita e crescimento ilimitado:** o feed das comunidades recebe novas postagens continuamente e nunca é reduzido, o que favorece um banco com escalabilidade horizontal.
- **Acesso por uma chave única de consulta**: o feed é sempre lido por comunidade e data (UC09) e o checklist, por vestibulando e dia (UC05), o que é atendido por um único índice composto, sem junções.
- **Esquema flexível**: as postagens podem ganhar novos atributos (reações, comentários, anexos) sem alteração de tabelas, mantendo a validação dos campos atuais por meio de $jsonSchema.
- **Sem dependência transacional:** nenhuma das duas entidades participa de regras que exijam transação entre várias tabelas (como o limite de vagas das sessões), por isso a atomicidade por documento do MongoDB é suficiente.

Persistência poliglota: o que fica em cada banco

Dado	Banco	Motivo
Usuário, Vestibulando, Psicólogo, Administrador	MySQL	Generalização, chaves candidatas (e-mail, CRP) e gatilhos de coerência (RN01, RN04)
Sessão e inscrições	MySQL	Limite de vagas e validação do psicólogo (RN02) exigem transação, CHECK e gatilho
Comunidade e participação	MySQL	Relacionamento N:M com integridade referencial
Desafio, Calendário e Rotina	MySQL	Dados pequenos, relacionais e com restrições declarativas
Postagem (feed)	**MongoDB**	Alto volume de escrita, leitura por comunidade e data
Checklist diário	**MongoDB**	Itens embutidos em um documento por vestibulando por dia

11.3. Mapeamento do Modelo Relacional para Coleções

O projeto lógico NoSQL parte do esquema relacional (seção 7), das cardinalidades do Diagrama de Classes e dos padrões de acesso dos casos de uso. Dois critérios orientaram cada decisão: os dados são **embutidos** (embedding) quando pertencem a um único documento, são lidos em conjunto e têm cardinalidade limitada; são **referenciados** (por _id) quando a coleção cresce sem limite ou é consultada de forma independente. A tabela a seguir relaciona cada tabela do esquema relacional à coleção correspondente.

Tabelas Relacionais	Coleção	Estratégia	Justificativa
postagem	postagens	Um documento por postagem; referências lógicas ao MySQL: autorId (usuario.id_usuario) e comunidadeId (comunidade.tipo)	Cresce sem limite e é consultada por comunidade e data (feed, UC09)
Checklist	Checklists	Um documento por vestibulando por dia, com os itens embutidos (itens[]); vestibulandoId referencia vestibulando no MySQL	Os itens do dia são lidos e gravados em conjunto (UC05); reduz o número de registros

Convenções adotadas na conversão:

Esquema Relacional (MySQL)	MongoDB
id_* INT UNSIGNED AUTO_INCREMENT (PK)	_id do tipo ObjectId, gerado pelo driver (o MongoDB não possui autoincremento)
fk_id_* INT UNSIGNED (FK)	Campo ObjectId com o nome do papel: psicologoId, vestibulandoId, autorId (postagem.fk_id_usuario)
VARCHAR(n)	string com maxLength n
ENUM(...)	string com enum no validador
TINYINT(1)	Bool
DATETIME	date, armazenada em UTC (horário de Brasília = UTC−3)
DATE	string "AAAA-MM-DD" (pattern), evitando deslocamento de fuso
snake case	camelCase (ex.: data_criacao → dataCriacao; status_validacao →statusValidacao)

Ajustes em relação ao esquema relacional: Em decorrência do modelo de documentos, foram feitos os ajustes abaixo:

- **checklists:** a chave candidata do MySQL (fk_id_vestibulando, data, item_descricao) passou a ser (vestibulandoId, data), pois os itens do dia ficam dentro do mesmo documento; a unicidade do item dentro do dia é garantida pela operação adicionarItemChecklist (seção 11.5) e o validador impede itens idênticos no array (uniqueItems).
- **postagens:** o atributo fk_id_comunidade (inteiro) foi substituído por comunidadeId, que guarda o próprio tipo da comunidade, permitindo filtrar o feed sem consultar o MySQL.

11.4. Decisões de Projeto Físico

- **Escopo:** somente Postagem e Checklist são armazenadas no MongoDB; as demais entidades permanecem no MySQL (seção 11.2).
- **Chaves primárias:** _id ObjectId (12 bytes), imutável e indexado automaticamente. A coleção comunidades usa chave natural (_id = tipo).
- **Chaves compostas:** o MongoDB não declara PK composta; o equivalente são os índices únicos compostos nomeados uk_* (tabela de índices abaixo). Optou-se por *não* usar _id composto, porque o _id é imutável e consultas por parte dele (ex.: todos os registros de um vestibulando) não aproveitam o índice _id. As PKs compostas de participa_sessao e participa_comunidade foram embutidas nos arrays inscricoes[] e comunidades[]; como um índice único não impede repetição entre elementos de um mesmo array, a unicidade da dupla é garantida pelas operações da seção 11.5 (filtro $ne no próprio documento).
- **Chaves candidatas (UK):** índices únicos nomeados uk_*. Para campos opcionais (psicologo.crp e administrador.credenciamento) usam-se índices únicos *parciais*, pois a ausência do campo seria tratada como valor nulo repetido nos demais documentos.
- **Índices de apoio:** idx_* atende à consulta principal do feed (comunidade e data, mais recentes primeiro). O índice único composto de checklists também atende à consulta por prefixo (todos os checklists de um vestibulando).
- **Integridade referencial entre bancos:**não há FOREIGN KEY entre o MongoDB e o MySQL. autorId, comunidadeId e vestibulandoId são referências lógicas, validadas pela API antes da gravação. O ON DELETE CASCADE é reproduzido pela operação excluirDadosNoSQL (seção 11.6): a API exclui o usuário no MySQL (cascata nas tabelas relacionais) e, em seguida, remove suas postagens e checklists no MongoDB. A operação é idempotente e pode ser repetida em caso de falha; enquanto não concluída, documentos órfãos não são exibidos, pois o autor é resolvido no MySQL.

- **Restrições:** validadores $jsonSchema e $expr por coleção, com validationLevel strict e validationAction error (escritas inválidas são rejeitadas), conforme a tabela de restrições abaixo.
- **Valores padrão:** : o MongoDB não possui DEFAULT. Os padrões do MySQL (tipo = "texto", realizado = false, dataCriacao atual) são definidos pela API ao inserir, e os campos correspondentes são obrigatórios nos validadores.
- **Segurança:**: o usuário de aplicação recarregavest_app tem apenas o papel readWrite no banco, sem privilégios administrativos; o servidor roda com authorization habilitado.
- **Consistência:** operações em um único documento são atômicas, o que atende às duas entidades; as gravações usam writeConcern majority quando há replica set.

Índices criados (além do índice _id_, criado automaticamente em cada coleção):

Índice	Coleção	Chave	Tipo	Origem / Fidelidade
uk_postagens_autor_comunidade_data	Postagens	{autorId: 1, comunidadeId: 1, dataCriacao: 1}	Único composto	Equivale a uk_postagem
idx_postagens_comunidade_data	Postagens	{comunidadeId: 1, dataCriacao: -1}	Composto	Feed da comunidade
uk_checklists_vestibulando_data	Checklists	{vestibulandoId: 1, data: -1}	Único composto	Um checklist por dia

Equivalência entre as restrições do esquema relacional e do MongoDB:

Restrição/ Recurso no MySQL	Equivalente no MongoDB
NOT NULL	required no $jsonSchema
UNIQUE (uk_*)	Índice único composto (uk_*)
CHECK texto não vazio (UC09 FE01)	minLength: 1 e pattern "\\S" em postagens.texto
CHECK booleano em {0,1}	bsonType: "bool"
CHECK formato da data	pattern "AAAA-MM-DD" em checklists.data
ENUM (tipo da comunidade)	enum em postagens.comunidadeId
uk_checklist (vestibulando, data, item)	Índice (vestibulandoId, data) + uniqueItems + operação adicionarItemChecklist
FOREIGN KEY / ON DELETE CASCADE	Referência lógica ao MySQL + operação excluirDadosNoSQL() na API
CREATE USER / GRANT	createUser com o papel readWrite no banco

11.5. Script de Criação do Esquema Físico

11.5.1. Configurações do Servidor

A configuração abaixo define o diretório de dados, a separação física por banco e por tipo de arquivo (collection/ e index/), a autenticação e o replica set.

```
# /etc/mongod.conf - RecarregaVest
storage:
  dbPath: /var/lib/mongodb
  directoryPerDB: true                  # um diretório por banco de dados
  wiredTiger:
       engineConfig:
       directoryForIndexes: true        # separa collection/ e index/
       collectionConfig:
       blockCompressor: snappy
systemLog:
  destination: file
  path: /var/log/mongodb/mongod.log
  logAppend: true
net:
  port: 27017
  bindIp: 127.0.0.1
  # em producao: tls: { mode: requireTLS, certificateKeyFile: <arquivo .pem> }
security:
  authorization: enabled
replication:
  replSetName: rs0


# Criacao do esquema, indices, view, usuario de aplicacao e carga inicial:
mongosh --file recarregavest_nosql.js

Observação: com authorization habilitado, o script deve ser executado por um
usuário com privilégios de administração do banco (por exemplo, o administrador
criado no banco admin).
```

11.5.2. Script DDL (coleções, validadores, índices, view e usuário de aplicação)

```
const rv = db.getSiblingDB("recarregavest_nosql");
rv.dropDatabase();

// Tipos (enum) das comunidades tematicas - RF05, UC10, UC11
const TIPOS_COMUNIDADE = [
  "ciencias_naturais", "humanas", "saude", "exatas", "estudo_trabalho",
  "geral", "noticias", "psicologos", "sessoes"];

// Padroes reutilizados nos validadores
const PAD_DATA = "^[0-9]{4}-[0-9]{2}-[0-9]{2}$";                          //
AAAA-MM-DD
```

```
// ---------------------------------------------------------------------
// POSTAGENS (feed das comunidades)
// ---------------------------------------------------------------------
rv.createCollection("postagens", {
  validationLevel: "strict",
  validationAction: "error",
  validator: { $jsonSchema: {
       bsonType: "object",
       required: ["autorId", "comunidadeId", "texto", "tipo", "dataCriacao"],
       additionalProperties: false,
       properties: {
       _id: { bsonType: "objectId" },
       autorId: { bsonType: "objectId", description: "Referencia usuarios._id" },
       comunidadeId: { enum: TIPOS_COMUNIDADE, description: "Referencia
comunidades._id" },
       texto: { bsonType: "string", minLength: 1, maxLength: 1000, pattern: "\\S",
              description: "Texto nao vazio (UC09 FE01)" },
       tipo: { bsonType: "string", minLength: 1, maxLength: 45 },
       urlExterna: { bsonType: ["string", "null"], maxLength: 255 },
       dataCriacao: { bsonType: "date" }
       }
  } }
});

 // ---------------------------------------------------------------------
// CHECKLISTS (um documento por vestibulando por dia, itens embutidos)
// vestibulandoId referencia vestibulando.fk_id_usuario no MySQL
// ---------------------------------------------------------------------
rv.createCollection("checklists", {
  validationLevel: "strict",
  validationAction: "error",
  validator: { $jsonSchema: {
       bsonType: "object",
       required: ["vestibulandoId", "data", "itens"],
       additionalProperties: false,
       properties: {
       _id: { bsonType: "objectId" },
       vestibulandoId: { bsonType: "int", minimum: 1 },
       data: { bsonType: "string", pattern: PAD_DATA },
       itens: {
       bsonType: "array", minItems: 1, uniqueItems: true,
       items: {
       bsonType: "object", additionalProperties: false,
       required: ["descricao", "realizado"],
       properties: {
              descricao: { bsonType: "string", minLength: 1, maxLength: 150 },
              realizado: { bsonType: "bool" }
       }
       }
       }



// ---------------------------------------------------------------------
// USUARIO DE APLICACAO (privilegio minimo)
// ---------------------------------------------------------------------
if (!rv.getUser("recarregavest_app")) {
  rv.createUser({ user: "recarregavest_app", pwd: passwordPrompt(),
```

```
roles: [{ role: "readWrite", db: "recarregavest_nosql" }] });
}
```

11.6. Operações de Integridade (substituem gatilhos e chaves estrangeiras)

Como o MongoDB não possui chaves estrangeiras e não impede itens repetidos dentro de um mesmo array, essas regras são implementadas em duas operações executadas pela API. As funções fazem parte do script entregue.

- **adicionarItemChecklist:** uma única operação updateOne com upsert, cujo filtro exige que o item ainda não exista no documento do dia; o $push e a criação do documento ocorrem juntos. Se o item já existe, o filtro não casa e o índice único (vestibulandoId, data) rejeita a criação de um segundo documento; a API trata o erro de chave duplicada como item repetido.
- **excluirDadosNoSQL:** reproduz o ON DELETE CASCADE entre os bancos. Após a exclusão do usuário no MySQL, a API remove as postagens e os checklists dele no MongoDB. Como não há transação distribuída, a operação é idempotente e deve ser repetida em caso de falha.

```
function adicionarItemChecklist(vestibulandoId, data, descricao) {
  return rv.checklists.updateOne(
       { vestibulandoId: NumberInt(vestibulandoId), data: data,
       "itens.descricao": { $ne: descricao } },
       { $push: { itens: { descricao: descricao, realizado: false } } },
       { upsert: true });
}

function excluirDadosNoSQL(idUsuario) {
  const id = NumberInt(idUsuario);
  rv.postagens.deleteMany({ autorId: id });
  rv.checklists.deleteMany({ vestibulandoId: id });
}

// Fluxo da API: (1) DELETE FROM usuario WHERE id_usuario = ?   (MySQL, cascata)
//            (2) excluirDadosNoSQL(id)                        (MongoDB)
```

11.7. Carga Inicial do Banco de Dados

A carga inicial contém as 6 postagens e os 6 itens de checklist (agrupados em 3 documentos) da massa de dados do DML relacional (seção 10.4). Os ids dos autores e dos vestibulandos são os mesmos ids inteiros dos usuários no MySQL, e os horários de Brasília do DML foram convertidos para UTC (ISODate). Os campos inteiros usam NumberInt, para que o mongosh não os grave como double e o validador os aceite.

```
// POSTAGENS (FEED)
rv.postagens.insertMany([
  { autorId: oid(2), comunidadeId: "psicologos",
       texto: "Bem-vindos à comunidade de psicólogos voluntários do RecarregaVest!",
       tipo: "texto", urlExterna: null,
       dataCriacao: ISODate("2026-09-21T13:00:00Z") },
  { autorId: oid(2), comunidadeId: "sessoes",
       texto: "Abrimos uma nova sessão sobre ansiedade. Inscrevam-se!",
       tipo: "link", urlExterna: "https://meet.example.com/ansiedade-1",
       dataCriacao: ISODate("2026-09-28T12:00:00Z") },
  { autorId: oid(6), comunidadeId: "saude",
       texto: "Alguém tem dicas para revisar Biologia em pouco tempo?",
       tipo: "texto", urlExterna: null,
       dataCriacao: ISODate("2026-09-29T23:15:00Z") },
  { autorId: oid(8), comunidadeId: "geral",
       texto: "Sugestão de playlist para relaxar antes de dormir.",
       tipo: "link", urlExterna: "https://open.spotify.com/",
       dataCriacao: ISODate("2026-10-01T01:00:00Z") },
  { autorId: oid(7), comunidadeId: "exatas",
       texto: "Resolvi 20 questões de Física hoje, foi difícil mas valeu!",
       tipo: "texto", urlExterna: null,
       dataCriacao: ISODate("2026-10-01T21:40:00Z") },
  { autorId: oid(3), comunidadeId: "estudo_trabalho",
       texto: "Lembrem de fazer pausas: 50 minutos de estudo, 10 de descanso.",
       tipo: "texto", urlExterna: null,
       dataCriacao: ISODate("2026-10-02T11:00:00Z") }
]);

// CHECKLIST DE AUTOCUIDADO (um documento por vestibulando por dia)
rv.checklists.insertMany([
  { vestibulandoId: oid(6), data: "2026-10-05", itens: [
       { descricao: "Beber 2 litros de água", realizado: false },
       { descricao: "Dormir pelo menos 7 horas", realizado: false },
       { descricao: "Caminhar 20 minutos", realizado: false }
  ] },
  { vestibulandoId: oid(7), data: "2026-10-05", itens: [
       { descricao: "Beber 2 litros de água", realizado: true },
       { descricao: "Dormir pelo menos 7 horas", realizado: true }
  ] },
  { vestibulandoId: oid(8), data: "2026-10-05", itens: [
       { descricao: "Fazer uma pausa sem telas", realizado: false }
  ] }
]);

// UPDATES DE USO
rv.checklists.updateOne({ vestibulandoId: oid(6), data: "2026-10-05" },
                        { $set: { "itens.$[i].realizado": true } },
                        { arrayFilters: [{ "i.descricao": "Beber 2 litros de água" }] });
rv.desafios.updateOne({ vestibulandoId: oid(7), data: "2026-10-05" },
                        { $set: { concluido: true } });
```

11.7.1. Consultas de verificação

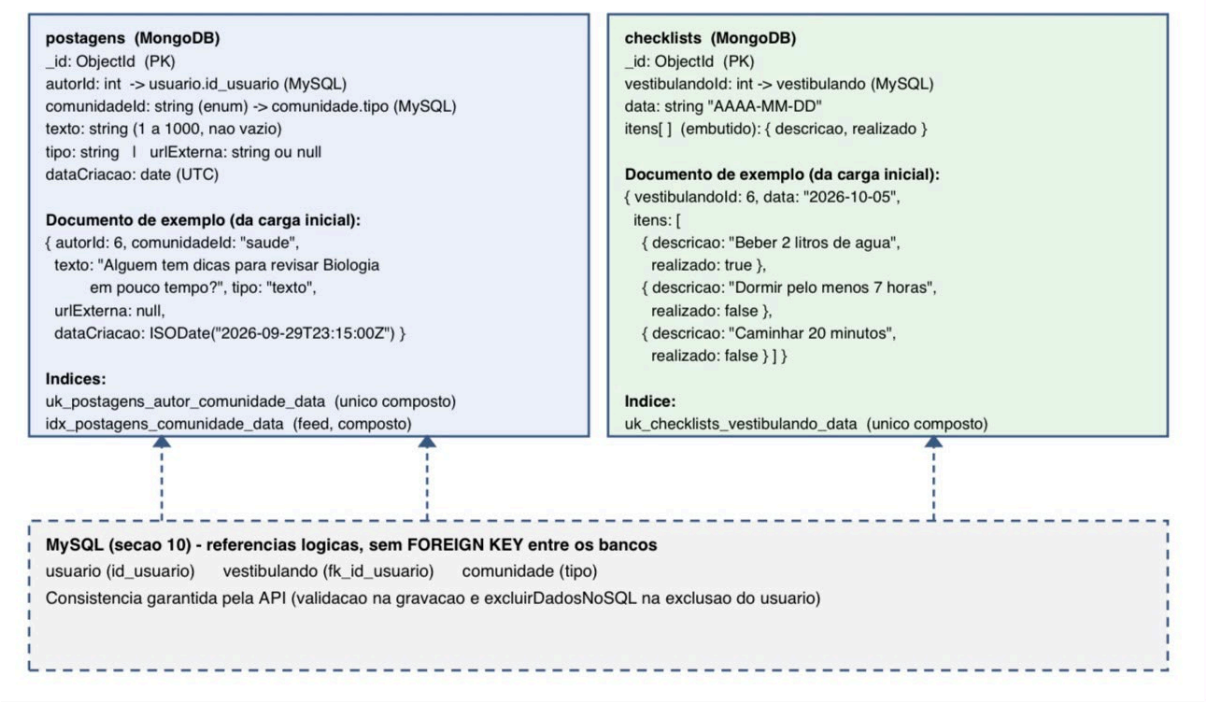
As consultas a seguir verificam a carga e o uso dos índices.

```
// 1) Feed da comunidade "saude", mais recentes primeiro (idx_postagens_comunidade_data)
rv.postagens.find({ comunidadeId: "saude" }).sort({ dataCriacao: -1 }).limit(20);

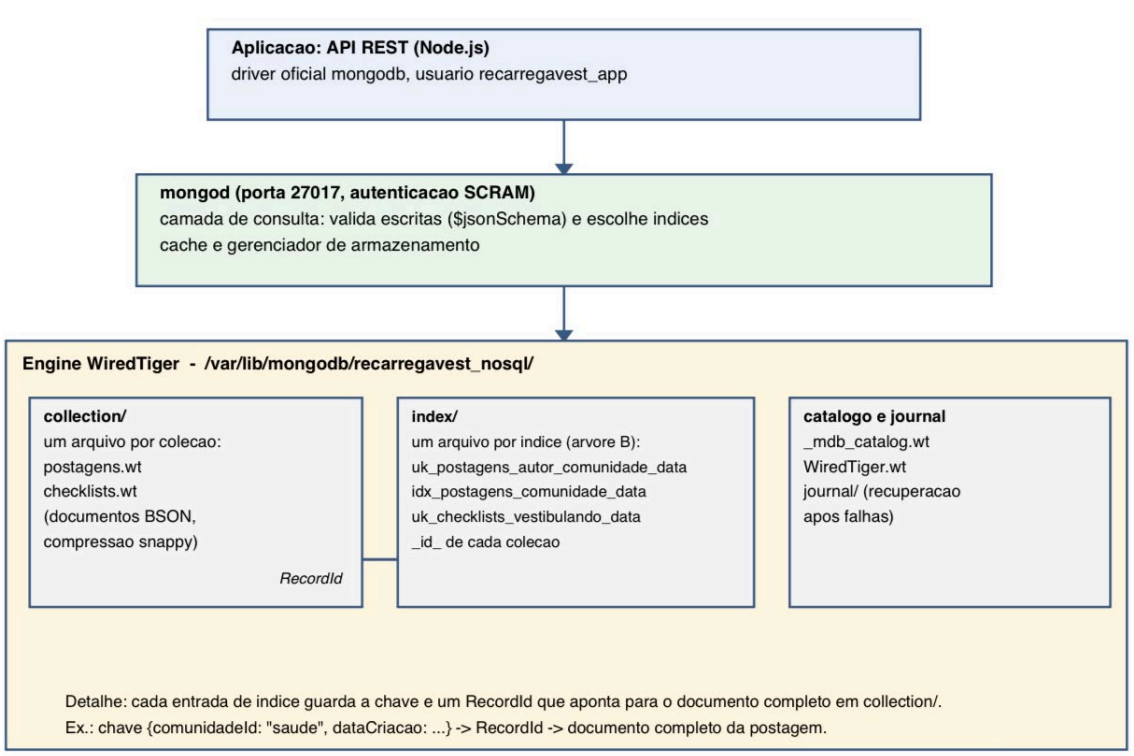
// 2) Checklist e desafio de um vestibulando em um dia (uk_checklists_vestibulando_data)
rv.checklists.findOne({ vestibulandoId: oid(6), data: "2026-10-05" });
rv.desafios.findOne({ vestibulandoId: oid(6), data: "2026-10-05" });
// 3) Confirmacao de que a consulta 1 usa o indice (esperado: IXSCAN)
rv.postagens.find({ comunidadeId: "saude" }).sort({ dataCriacao: -1
}).explain("executionStats");
```

11.8. Diagramas da Estrutura Física dos Dados

A Figura apresenta as duas coleções do MongoDB, cada uma com um documento de exemplo extraído da carga inicial, os campos embutidos, as referências lógicas para o MySQL, as chaves e os índices de cada coleção.



A Figura mostra a organização física desses dados no SGBD: a aplicação se conecta ao servidor mongod; a camada de consulta valida as escritas e escolhe os índices; o engine WiredTiger mantém um arquivo por coleção (documentos BSON) e um arquivo por índice (árvore B), além do catálogo e do journal; e, em detalhe, cada entrada de índice aponta por RecordId para o documento completo na coleção.


Aplicacao: API REST (Node.js)
driver oficial mongodb, usuario recarregavest_app
mongod (porta 27017, autenticacao SCRAM)
camada de consulta: valida escritas ($jsonSchema) e escolhe indices
cache e gerenciador de armazenamento
Engine WiredTiger - /var/lib/mongodb/recarregavest_nosql/
collection/
um arquivo por colecao:
postagens.wt
checklists.wt
(documentos BSON,
compressao snappy)
RecordId
index/
um arquivo por indice (arvore B):
uk_postagens_autor_comunidade_data
idx_postagens_comunidade_data
uk_checklists_vestibulando_data
_id_ de cada colecao
catalogo e journal
_mdb_catalog.wt
WiredTiger.wt
journal/ (recuperacao
apos falhas)
Detalhe: cada entrada de indice guarda a chave e um RecordId que aponta para o documento completo em collection/.
Ex.: chave {comunidadeId: "saude", dataCriacao: ...} -> RecordId -> documento completo da postagem.